



30

Implémentation du flow

US3.4 · C6/C7

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Implémenter les 4 tâches sur les vraies données
- › Persister les prédictions (table predictions)
- › Garantir l'idempotence (upsert, relance sans doublon)
- › Diagnostiquer la cible date seule vs date+time

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE flow end-to-end · 2 runs → 0 doublon · panne 4,78 %

1

Module 30 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Le flow a planté en milieu de course puis a été relancé. Sans idempotence, tu te retrouves avec des doublons dans la base — et des statistiques faussées.

Angle éthique / risque (C2) Des doublons = des décisions biaisées. La clé d'unicité garantit l'intégrité des données (lien C3).

■ Contexte & outils — pourquoi ce module

Contexte : on passe du design au code du flow. Pourquoi : écrire dans la base sans jamais dupliquer une ligne.

- SQLAlchemy / psycopg : accès à la base depuis Python
- upsert ON CONFLICT : insère OU met à jour (idempotence prouvée)
- TIMESTAMPTZ en UTC : des horodatages comparables
- Clé naturelle : identifie une ligne de manière unique

Objectif du module Deux exécutions du flow -> zéro doublon en base.

■ Historiser les prédictions : la table predictions

Chaque prédiction est tracée en base, avec une clé naturelle qui empêche les doublons.

- Clé (machine, prediction_ts) ; on réutilise la lib (build_dataset, predict_proba)
- Historiser permet ensuite de surveiller le modèle (drift, dashboards)

! À retenir

Le flow ne réimplémente RIEN : il appelle les fonctions déjà testées du package. Moins de code = moins de bugs.

i Le jargon décodé

Clé naturelle = colonnes qui identifient une ligne de façon unique. Historisation = garder la trace de chaque prédiction.

EN UNE PHRASE On orchestre la lib existante et on historise chaque prédiction.

■ L'idempotence par upsert

Rejouer le flow ne doit jamais dupliquer : on met à jour au lieu d'insérer.



! Attention — piège

INSERT simple = doublons garantis à la 2e exécution. La preuve d'idempotence (count(*) stable) est non négociable.

+ Astuce de pro

La task store touche la DB (ressource externe) → on lui met des retries ; les tasks de calcul pur, non.

EN UNE PHRASE Upsert (ON CONFLICT ... DO UPDATE) → 2 runs = 0 doublon.

■ Le piège data : la cible « date » seule

Oublier l'heure de l'incident décale la fenêtre et fausse silencieusement la prévalence des pannes.

date seule → 4,8914 %

- heure = minuit
- 3 210 pannes
- +73 faux positifs

date + time → 4,7802 %

- horodatage exact
- 3 137 pannes
- = référence 95

→ Cas réel InduSense

Symptôme : `panne.mean()` $\approx$ 4,89 %. Diagnostic : `incident_ts.dt.time` à minuit. Correction : `date+time`. Retour à 4,78 %.

? Le saviez-vous ?

+0,11 point de prévalence paraît minuscule... mais ce sont 73 fausses alarmes qui dégradent la confiance dans le modèle.

EN UNE PHRASE En prod, le diable est dans le time : toujours combiner date + time.

■ L'upsert, en vrai (idempotence + horodatage)

On historise dans predictions, clé (machine, prediction_ts) ; rejouer ne duplique pas.

```
INSERT INTO predictions (machine, prediction_ts, proba_panne, decision, written_at)
VALUES (...)
ON CONFLICT (machine, prediction_ts) DO UPDATE SET
  proba_panne = EXCLUDED.proba_panne, decision = EXCLUDED.decision,
  written_at = EXCLUDED.written_at;
-- 2 exécutions -> count(*) IDENTIQUE (0 doublon)
-- horodatage incident : pd.to_datetime(date + ' ' + time)
```



Le saviez-vous ?

date seule (minuit) -> 4,8914 % (+73 FP) ; date+time -> 4,7802 % (jeu complet ;
echantillon data/sample ~10,5 %).



Cas réel InduSense

store touche la DB (ressource externe) -> retries ; les tasks de calcul pur, non.

EN UNE PHRASE Preuve d'idempotence : 2 runs -> même count(*). C6/C7.

■ À retenir en 3 points

- L'upsert évite les doublons.
- Deux runs $\rightarrow$ count(*) stable : c'est la preuve d'idempotence.
- Un flow idempotent se rejoue sans dégât.

2

Module 30 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — predict & store (upsert)

```
INSERT INTO predictions(machine,prediction_ts,proba_panne,decision,written_at)
VALUES(...) ON CONFLICT (machine,prediction_ts)
DO UPDATE SET proba_panne=EXCLUDED.proba_panne, decision=EXCLUDED.decision, written_at=EXCLUDED.written_at;
```

store a des retries C'est une ressource externe (DB) → pannes transitoires possibles.

■ TP — preuve d'idempotence

```
# PostgreSQL · Compose @db:5432 · définir $sourceData · exécuter 2×  
docker compose run --rm --no-deps -e PREFECT_PROFILE=ephemeral -e INDUSENSE_DATA_DIR=/d -v "${sourceData}:/d:ro" api python -m indusense.flows.predict_flow  
# Windows · SQLite : $env:INDUSENSE_DB_URL="sqlite:///predictions.db" ; uv run python -m indusense.flows.predict_flow # 2×
```

✅ **Preuve attendue** 2 exécutions → même count(*) (0 doublon) ; panne ≈ 4,78 %.

3

Module 30 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — enrichir le flow

Avancez à votre rythme. Le formateur reste disponible.

- Ajouter une colonne decision (proba > seuil) à la table + au store
- OU Prefect 3 : `predict_flow.serve(name="indusense-06h", cron="0 6 * * *")`

+ Posture en distanciel

Repli SQLite si pas de Postgres : même logique d'upsert. Montrez vos deux `count(*)` pour valider l'idempotence.

■ Definition of Done & transition

- DoD : flow exécutable, prédictions persistées, relance idempotente

Transition → Module 31 Le pipeline tourne : on surveille la dérive des données (drift).